F20FO - Digital Forensics Coursework 1

Shyam Sundar Velmurugan
BSc Computer Science (Cyber Security)
H00418621
ssv2001@hw.ac.uk
Video Link: F20F0 Cw1.mp4

Contents

Introduction 3
Windows VM 3
Wireshark 3
Autopsy 3
Linux VM (Ubuntu) 3
The Sleuth Kit 3
Foremost 3
Illustrative Examples 4
Controlled Virtual Network Environment 4
Windows VM (Wireshark) 5
Linux VM (The Sleuth Kit) 7
Limitations 12
Conclusion 12
Reference 13

Word Count - 1560

Introduction

This report presents the design and implementation of a structured digital forensics laboratory using virtual machines, including the deployment and analysis of selected forensic tool across Linux and Windows platforms.

Conducting network-based forensics was done with Wireshark to examine communications between packets and traffic patterns. Analysis on disks and filesystem were performed using The Sleuth Kit to demonstrate inspection on metadata, recovering inode, and reconstructing timeline. Through practical experimentations and technical evaluations, the report examines the capabilities, strengths, and limitations of these tools within a controlled environment.

On Windows virtual machine, the tools chosen are Wireshark and Autopsy, and For Linux virtual machine, the tools chosen are The Sleuth Kit and Foremost.

Windows VM

Wireshark

Wireshark was selected for its deep packet inspection and protocol analysis capabilities (Combs,2023). It enables filtering, TCP stream reconstruction, and statistical traffic analysis making it suitable for network forensic investigations.

Autopsy

Autopsy was selected as a graphical digital forensics platform to support structured disk analysis and case management (Carrier, B. (2023)). It provides an interface for examining file systems, recovering deleted files, and performing keyword searches. Overall, it enhances investigating workflow by organising evidence withing a structure, improving the documentation and efficient reporting.

Linux VM (Ubuntu)

The Sleuth Kit

Sleuth Kit was selected as a command-line digital forensics toolkit (Carrier, B. (2023)). This tool helps to examine disk image at inode and block level, recovering deleted files, and analysing file system structures without mounting evidence directly. Its metadata-based analytical approach makes it suitable for forensic investigations withing a controlled environment.

Foremost

To demonstrate data recovery independent of file system metadata, a file carving tool like Foremost was implemented (Martins de Souza et al. (2025)). It scans raw disk images for known file headers and footers, enabling recovery of files even when metadata structures are deleted or corrupted.

Illustrative Examples

Controlled Virtual Network Environment

Configured a controlled virtual networking environment for a secure and isolated communication between both virtual machines. This configuration was done in order to simulate an internal network communication while separating from external internet environment.

Within VirtualBox, choose any virtual machine and set up network settings by choosing NAT as adapter 1, and Host-Only Network as adapter 2. This was followed for the other virtual machine also. It was configured to create a private internal network shared exclusively between host system and virtual machines.

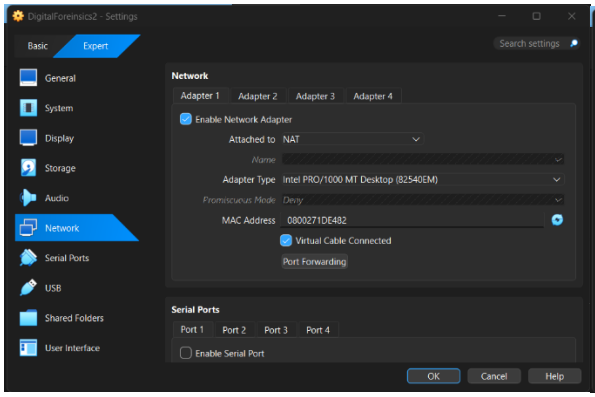


Figure 1: Setting Adapter 1 as NAT

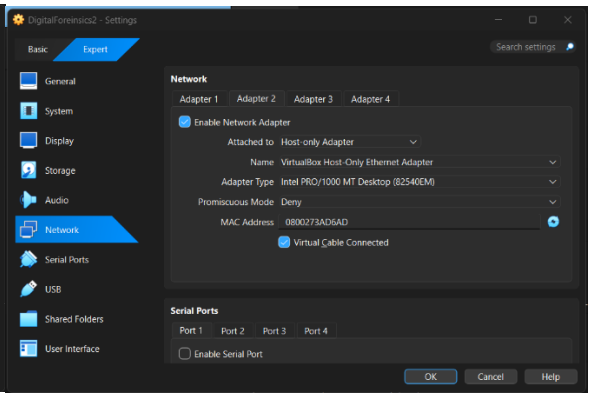


Figure 2 Setting Adapter 2 as Host-Only Adapter

Once this is done, turn on both the virtual machines. In Windows, open cmd and run the command ipconfig. With this we can get the IP address. In Linux, open the terminal and run the command ip a.

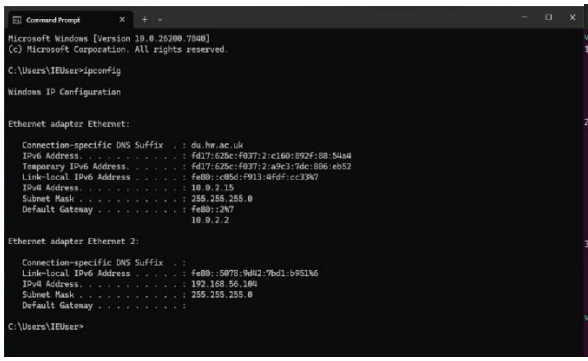


Figure 3 Running the command for Windows

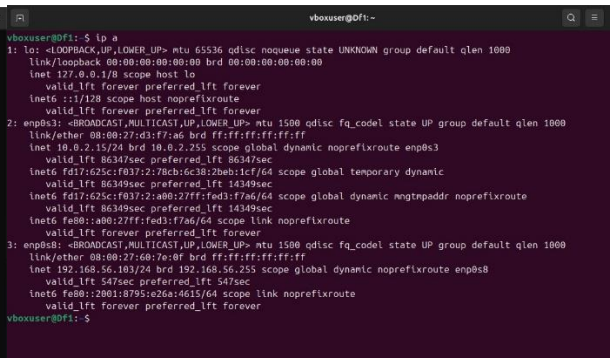


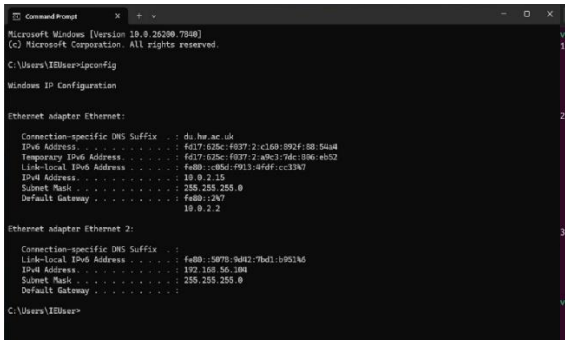
Figure 4 Running the command for Linux

After this, the network communication was validated using the ICMP echo requests by pinging between both systems. Initial test from Windows to Ubuntu was successful. But the pinging between requests from Ubuntu to Windows failed. This issue was identified as being caused by Windows Defender Firewall, which is responsible for blocking inbound ICMP requests by default. This can be sorted by

- Go to windows virtual machine.
- Open run or click windows + R

- Type wf.msc (opens windows defender firewall with advance security)
- Navigate into inbound rules and search for 'File and Printer Sharing (Echo Request - ICMPv4-In)' {Make sure the profile is private}
- Right click and enable that one.
- Retry pinging

After this, the bidirectional ping was successfully established between both virtual machines.



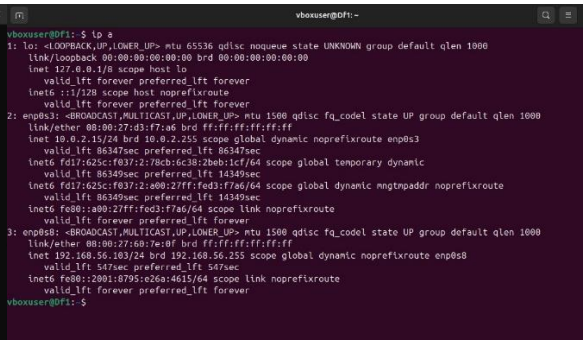
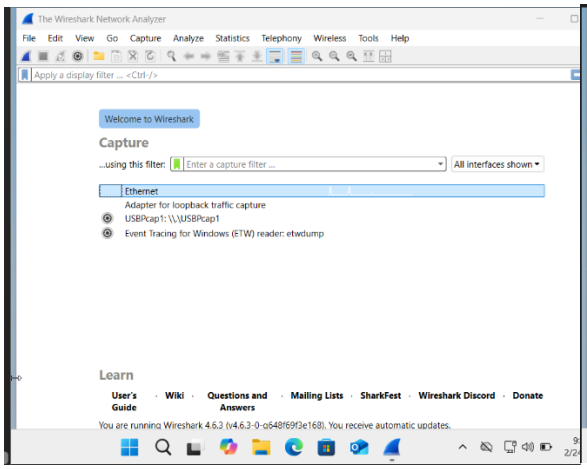


Figure 5 Running the command for windows *Figure 6 Running the command for Linux*

Windows VM (Wireshark)

Wireshark was used for capturing and analysing live network traffic generated within the virtual environment. After installing and setting up the windows x64 version with Npcap and WinPcap-compatible mode, capturing packets was initiated on the active Host-Only interface. Traffic was generated by visiting both HTTP and HTTPS website and performing ICMP communication using the ping command between virtual machines. For starting a capture, select the active ethernet adapter which has a graph towards its right side. Once it starts capturing, navigate to browser and visit two websites for 10 to 15 seconds and then stop capturing.



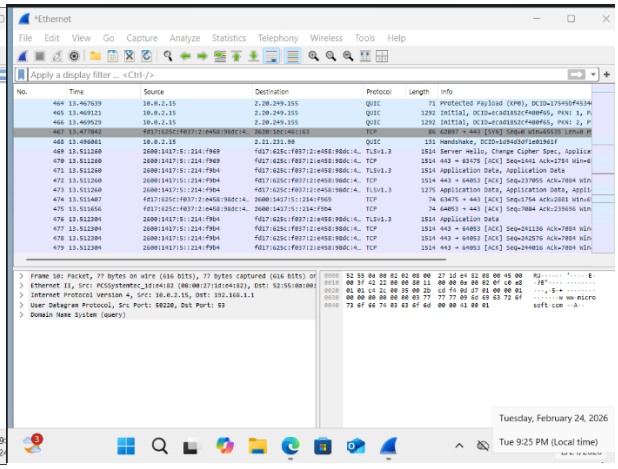


Figure 7 Interface of Wireshark *Figure 8 Showing all the packets*

The captured packets were examined using display filters such as dns, http, tcp, and icmp to isolate specific protocol activity. DNS queries and responses were analysed to reconstruct resolution behaviour of domain, while TCP streams were followed to observe full client-server communication sessions.

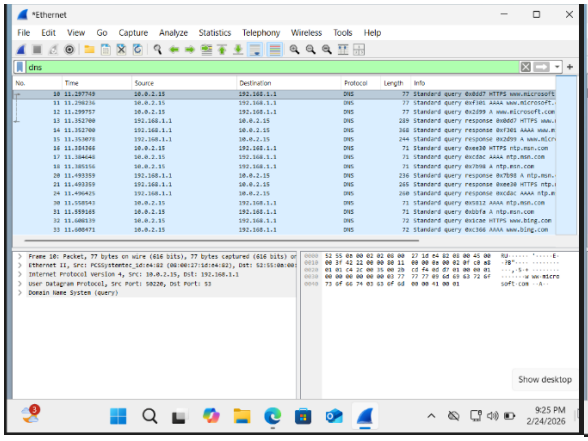


Figure 9 Showing only DNS packets

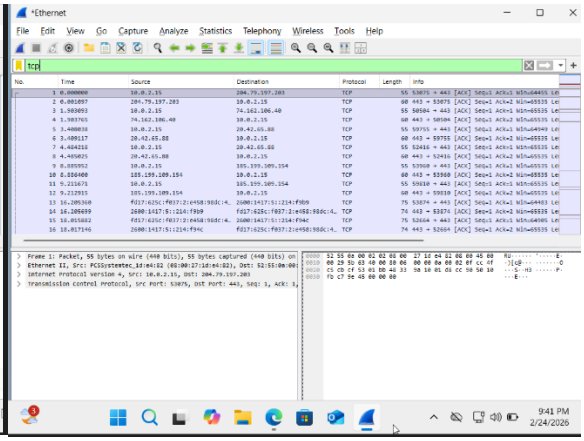


Figure 10 Showing only TCP packets

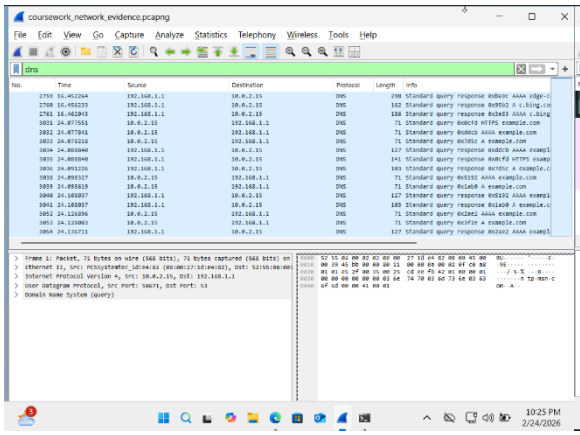


Figure 11 Filtering for DNS

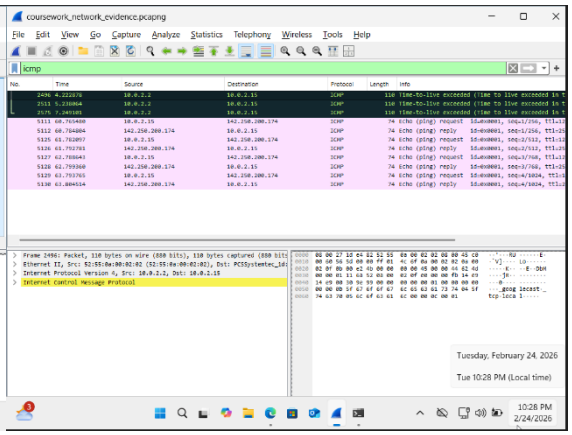


Figure 12 Filtering for ICMP

Advance filtering such as tcp.flag.syn ==1 was applied to identify the connection establishment packets and interpret flag values (below the Transmission Control Protocol, the Flags: 0x0012 (SYN,ACK)). This value is because of the numeric value provided by to TCP flags, where for ACK, its 0x10 and for SYN, its 0x02.

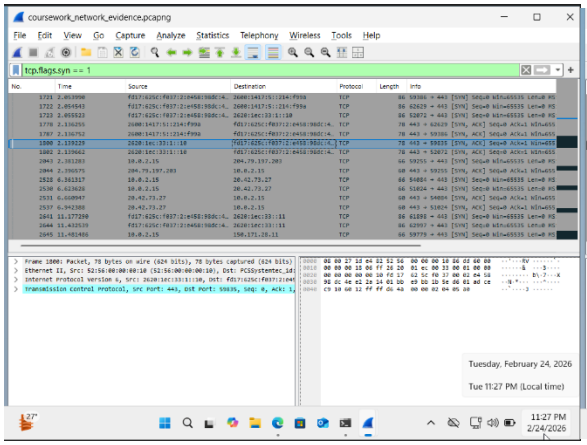


Figure 13 Advance displaying queries

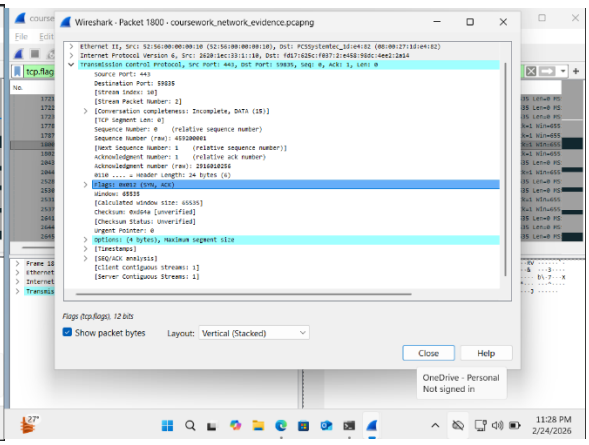


Figure 14 Proving the display queries

Statistical tools like Protocol Hierarchy, Conversations, and I/O graphs were used for visualising traffic distribution, communication endpoints, and activity bursts.

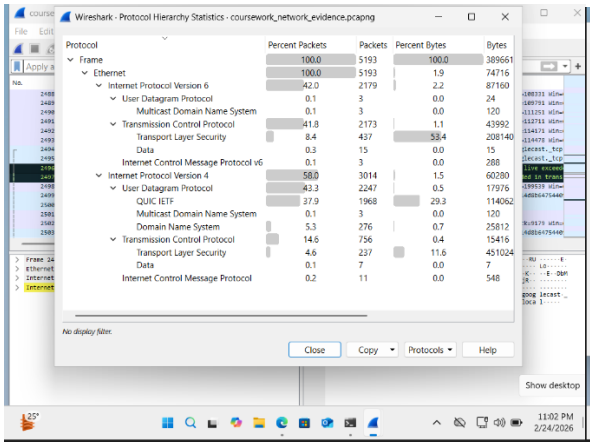


Figure 15 Protocol hierarchy

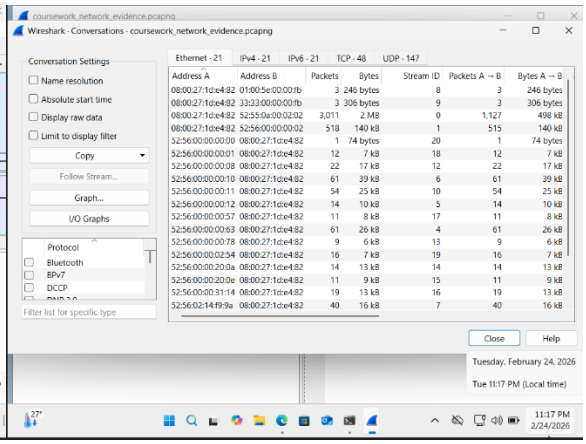


Figure 16 Conversations

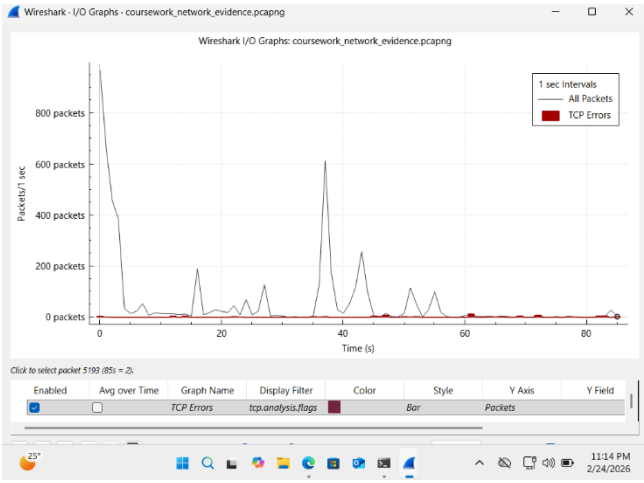


Figure 17 I/O Graph

A SHA-256 hash of the captured file was generated to ensure the evidence integrity, demonstrating forensic awareness and adherence to digital evidence handling principles. The use of SHA-256 hashing prior to analysis reflects adherence to principle of forensic soundness. In digital investigations, maintaining the integrity of evidence is critical to ensure reliability.

Linux VM (The Sleuth Kit)

Sleuth Kit was used to perform low-level forensic analysis of a custom-created disk image within Linux environment. The Linux version is checked and updated before installing the tool.

The EXT4 filesystem structure organises the data through superblocks, inodes and data blocks. Inodes store metadata like file size, ownership, timestamps, while the directory entries map filenames to inode numbers. While the files are deleted and lose the directory references, the inode number and block-level data is retained until overwritten.

```
vboxuser@Df1: ~
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

vboxuser@Df1:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.4 LTS
Release:        24.04
Codename:       noble
vboxuser@Df1:~$
```

Figure 18 Version check

```
vboxuser@Df1: ~
Codename:       noble
vboxuser@Df1:~$ sudo apt update -y
[sudo] password for vboxuser:
Hit:1 http://ae.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://ae.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,49
1 kB]
Get:5 http://ae.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://ae.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,8
00 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [239
kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21
.5 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata
 [9,932 B]
Get:10 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Package
s [2,557 kB]
Get:11 http://ae.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [33
1 kB]
Get:12 http://ae.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [
175 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-e
```

Figure 19 Linux updating

```
vboxuser@Df1: ~
vboxuser@Df1:~$ sudo apt install sleuthkit -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libafflib0t64 libbfio1 libdate-manip-perl libewf2 libtsk19t64 libvhdi1
  libvmdk1
Suggested packages:
  autopsy mac-robber
The following NEW packages will be installed:
  libafflib0t64 libbfio1 libdate-manip-perl libewf2 libtsk19t64 libvhdi1
  libvmdk1 sleuthkit
0 upgraded, 8 newly installed, 0 to remove and 25 not upgraded.
Need to get 3,722 kB of archives.
After this operation, 20.3 MB of additional disk space will be used.
Get:1 http://ae.archive.ubuntu.com/ubuntu noble/universe amd64 libafflib0t64 amd
64 3.7.20-1.1ubuntu5 [224 kB]
Get:2 http://ae.archive.ubuntu.com/ubuntu noble/universe amd64 libbfio1 amd64 20
170123-6 [306 kB]
Get:3 http://ae.archive.ubuntu.com/ubuntu noble/main amd64 libdate-manip-perl al
l 6.95-1 [923 kB]
Get:4 http://ae.archive.ubuntu.com/ubuntu noble/universe amd64 libewf2 amd64 201
40814-1build3 [573 kB]
Get:5 http://ae.archive.ubuntu.com/ubuntu noble/universe amd64 libvhdi1 amd64 20
```

Figure 20 Sleuth kit installation

```
vboxuser@Df1: ~
Preparing to unpack .../4-libvhdi1_20210425-1build5_amd64.deb ...
Unpacking libvhdi1:amd64 (20210425-1build5) ...
Selecting previously unselected package libvmdk1:amd64.
Preparing to unpack .../5-libvmdk1_20200926-2build5_amd64.deb ...
Unpacking libvmdk1:amd64 (20200926-2build5) ...
Selecting previously unselected package libtsk19t64:amd64.
Preparing to unpack .../6-libtsk19t64_4.12.1+dfsg-1.1ubuntu2_amd64.deb ...
Unpacking libtsk19t64:amd64 (4.12.1+dfsg-1.1ubuntu2) ...
Selecting previously unselected package sleuthkit.
Preparing to unpack .../7-sleuthkit_4.12.1+dfsg-1.1ubuntu2_amd64.deb ...
Unpacking sleuthkit (4.12.1+dfsg-1.1ubuntu2) ...
Setting up libvhdi1:amd64 (20210425-1build5) ...
Setting up libafflib0t64:amd64 (3.7.20-1.1ubuntu5) ...
Setting up libewf2:amd64 (20140814-1build3) ...
Setting up libdate-manip-perl (6.95-1) ...
Setting up libbfio1:amd64 (20170123-6) ...
Setting up libvmdk1:amd64 (20200926-2build5) ...
Setting up libtsk19t64:amd64 (4.12.1+dfsg-1.1ubuntu2) ...
Setting up sleuthkit (4.12.1+dfsg-1.1ubuntu2) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
vboxuser@Df1:~$ tsk_loaddb -V
The Sleuth Kit ver 4.12.1
vboxuser@Df1:~$
```

Figure 21 Check Sleuth kit version

After verifying system configurations and installing the toolset, a directory was created. Once that is done, the dd version was checked and 100MB raw disk image (testimage.dd) was generated and formatted with an EXT4 file system to simulate realistic storage structure. Later confirmed that the image exists.

```
vboxuser@Df1: ~/forensics
Selecting previously unselected package libvmdk1:amd64.
Preparing to unpack .../5-libvmdk1_20200926-2build5_amd64.deb ...
Unpacking libvmdk1:amd64 (20200926-2build5) ...
Selecting previously unselected package libtsk19t64:amd64.
Preparing to unpack .../6-libtsk19t64_4.12.1+dfsg-1.1ubuntu2_amd64.deb ...
Unpacking libtsk19t64:amd64 (4.12.1+dfsg-1.1ubuntu2) ...
Selecting previously unselected package sleuthkit.
Preparing to unpack .../7-sleuthkit_4.12.1+dfsg-1.1ubuntu2_amd64.deb ...
Unpacking sleuthkit (4.12.1+dfsg-1.1ubuntu2) ...
Setting up libvhdi1:amd64 (20210425-1build5) ...
Setting up libafflib0t64:amd64 (3.7.20-1.1ubuntu5) ...
Setting up libewf2:amd64 (20140814-1build3) ...
Setting up libdate-manip-perl (6.95-1) ...
Setting up libbfio1:amd64 (20170123-6) ...
Setting up libvmdk1:amd64 (20200926-2build5) ...
Setting up libtsk19t64:amd64 (4.12.1+dfsg-1.1ubuntu2) ...
Setting up sleuthkit (4.12.1+dfsg-1.1ubuntu2) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
vboxuser@Df1:~$ tsk_loaddb -V
The Sleuth Kit ver 4.12.1
vboxuser@Df1:~$ mkdir ~/forensics
vboxuser@Df1:~$ cd ~/forensics
vboxuser@Df1:~/forensics$
```

Figure 22 Creating a directory

```
vboxuser@Df1: ~/forensics
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
vboxuser@Df1:~$ tsk_loaddb -V
The Sleuth Kit ver 4.12.1
vboxuser@Df1:~$ mkdir ~/forensics
vboxuser@Df1:~$ cd ~/forensics
vboxuser@Df1:~/forensics$ dd if=/dev/zero of=testimage.dd bs=1M count =100
dd: unrecognized operand 'count'
Try 'dd --help' for more information.
vboxuser@Df1:~/forensics$ dd --version
dd (coreutils) 9.4
Copyright (C) 2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Written by Paul Rubin, David MacKenzie, and Stuart Kemp.
vboxuser@Df1:~/forensics$ dd if=/dev/zero of=testimage.dd bs=1M count=100
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.0235912 s, 4.4 GB/s
vboxuser@Df1:~/forensics$ ls -lh testimage.dd
-rw-rw-r-- 1 vboxuser vboxuser 100M Feb 25 20:46 testimage.dd
vboxuser@Df1:~/forensics$
```

Figure 23 Creation of image and checking dd version

The image was examined using the fssat for extracting filesystem metadata such as block size and inode information, confirming the proper filesystem creation. File system contents were enumerated using fls, which reveals directory structures including the lost+found, virtual entries, and internal EXT4 metadata like $OrphanFiles. Inode-level analysis was conducted

using istat for inspecting directory metadata, demonstrating detailed filesystem interrogation capability.

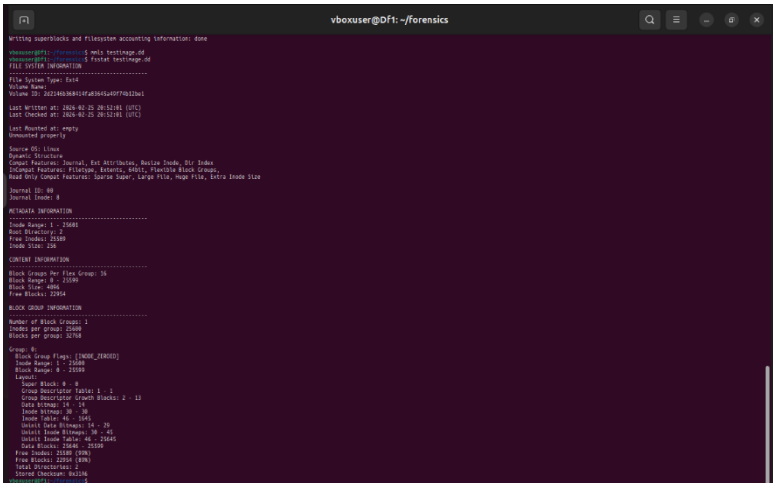
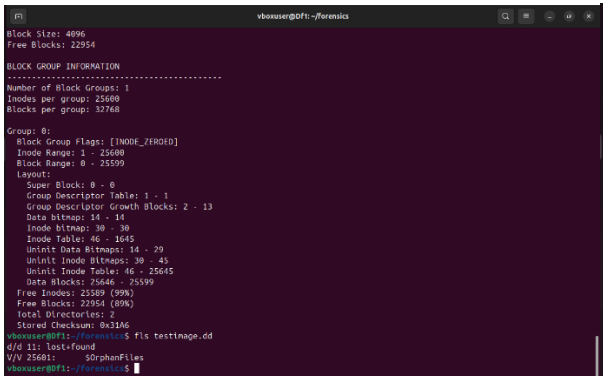


Figure 24 Examining file using fssat



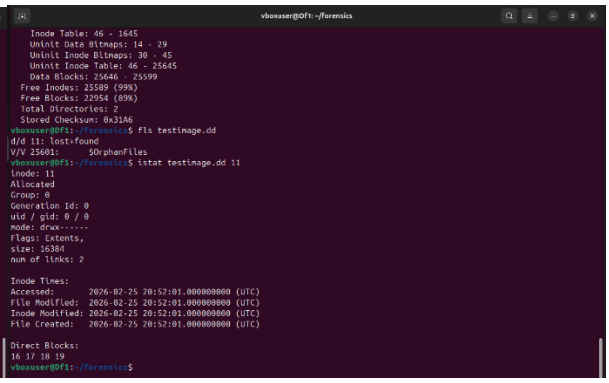


Figure 25 Using fls to reveal directory structure

Figure 26 Using istat for inspection

A SHA-256 hash of that image was calculated and preserved prior to further manipulation, just to ensure forensic integrity. Image was then mounted to create and modify a sample file (secret.txt) in a loopback mode, after which it was unmounted to maintain consistent evidentiary.

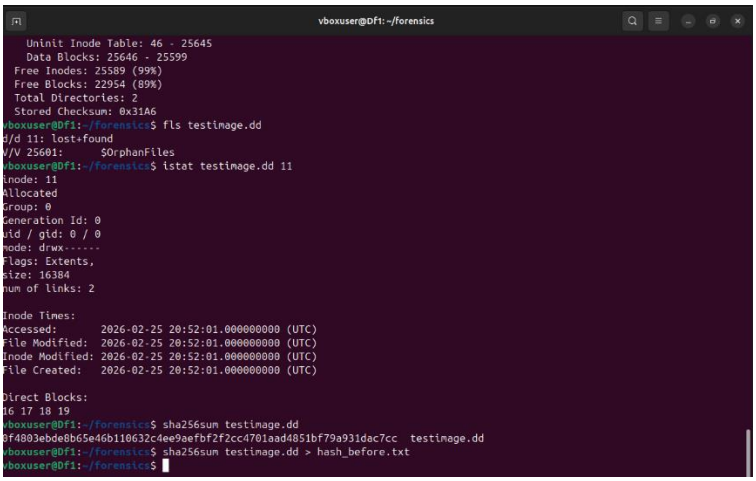


Figure 27 Converting image into SHA-256 and saving into .txt file

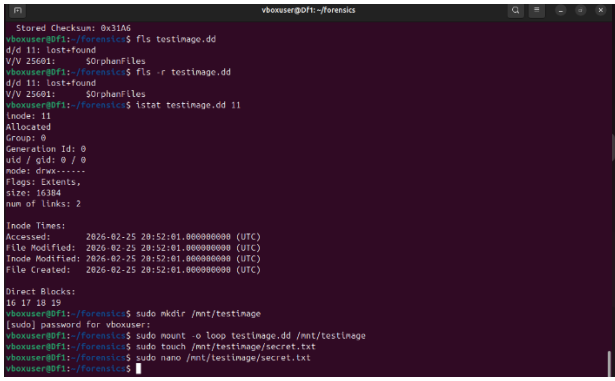


Figure 28 Creating mount directory, mounting it and adding text inside

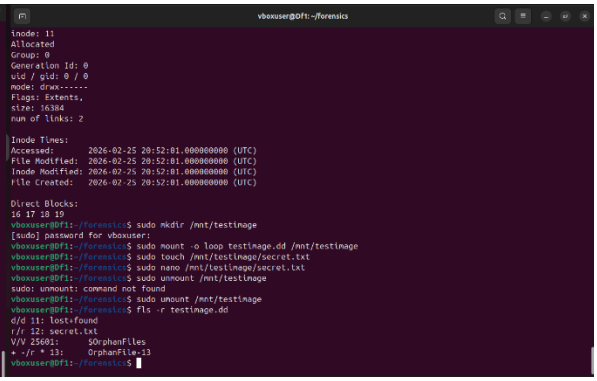


Figure 29 Fls command to show secret.txt file

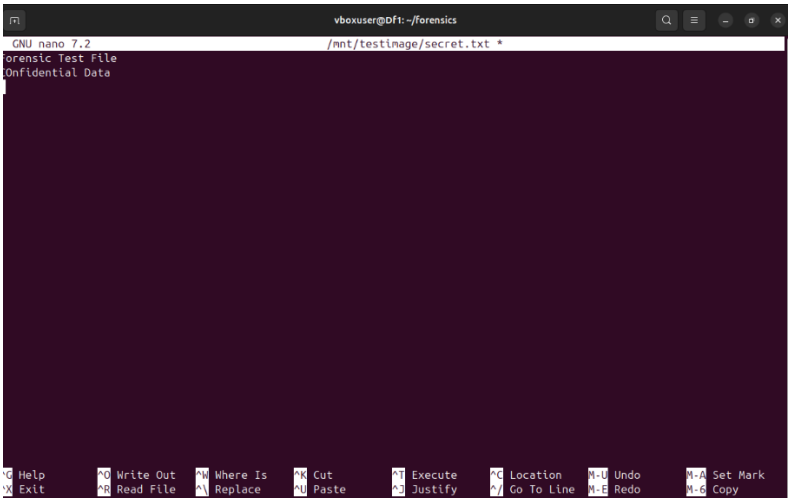


Figure 30 Adding some text within .txt file

File recovery was demonstrated using icat by extracting data directly from its inode, even after deletion. Finally, a forensic timeline was generated using fls to create a body file and the mactime was used to convert it into a chronological activity log, illustrating event reconstruction through MAC (Modified, Accessed, Changed) timestamp analysis.

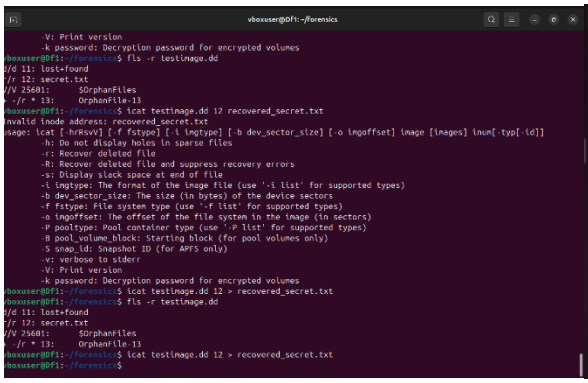


Figure 31 Finding inode number and recovering file

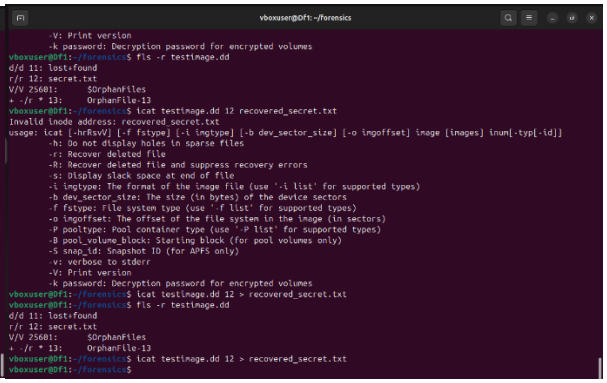


Figure 32 Cat command to recover the file

When secret.txt was deleted from the mounted image, it did not immediately erase it. Using fls, deleted file was identified with asterisk(*), saying that file had been remove but inode reference remained available.

```
vboxuser@Df1: ~/forensics
V/V 25601:      $OrphanFiles
+ -/r * 13:     OrphanFile-13
vboxuser@Df1:~/forensics$ icat testimage.dd 12 recovered_secret.txt
Invalid inode address: recovered_secret.txt
usage: icat [-hrRsvV] [-f fstype] [-i imgtype] [-b dev_sector_size] [-o imgoffset] image [images] inum[-typ[-id]]
        -h: Do not display holes in sparse files
        -r: Recover deleted file
        -R: Recover deleted file and suppress recovery errors
        -s: Display slack space at end of file
        -i imgtype: The format of the image file (use '-i list' for supported types)
        -b dev_sector_size: The size (in bytes) of the device sectors
        -f fstype: File system type (use '-f list' for supported types)
        -o imgoffset: The offset of the file system in the image (in sectors)
        -P pooltype: Pool container type (use '-P list' for supported types)
        -B pool_volume_block: Starting block (for pool volumes only)
        -S snap_id: Snapshot ID (for APFS only)
        -v: verbose to stderr
        -V: Print version
        -k password: Decryption password for encrypted volumes
vboxuser@Df1:~/forensics$ icat testimage.dd 12 > recovered_secret.txt
vboxuser@Df1:~/forensics$ fls -r testimage.dd
d/d 11: lost+found
r/r 12: secret.txt
V/V 25601:      $OrphanFiles
+ -/r * 13:     OrphanFile-13
vboxuser@Df1:~/forensics$ icat testimage.dd 12 > recovered_secret.txt
vboxuser@Df1:~/forensics$ cat recovered_secret.txt
Forensic Test File
COnfidential Data
vboxuser@Df1:~/forensics$ sudo mount -o loop testimage.dd /mnt/testimage
vboxuser@Df1:~/forensics$ sudo rm /mnt/testimage/secret.txt
vboxuser@Df1:~/forensics$
```

Figure 33 Deleting a file

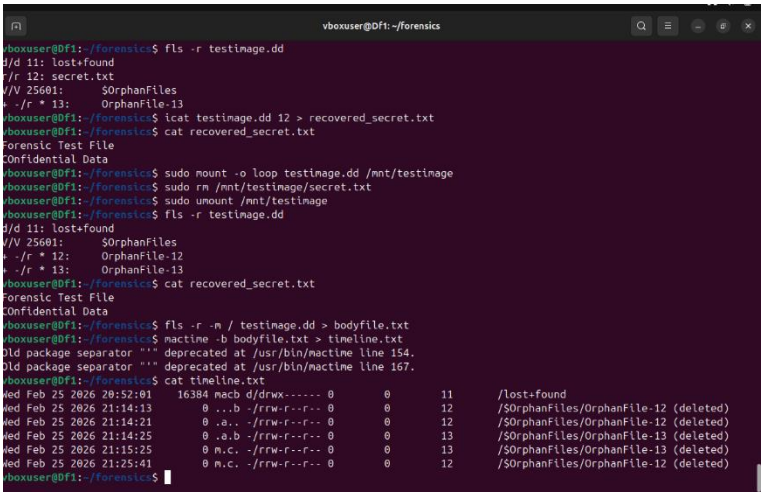
```
vboxuser@Df1: ~/forensics
        -i imgtype: The format of the image file (use '-i list' for supported types)
        -b dev_sector_size: The size (in bytes) of the device sectors
        -f fstype: File system type (use '-f list' for supported types)
        -o imgoffset: The offset of the file system in the image (in sectors)
        -P pooltype: Pool container type (use '-P list' for supported types)
        -B pool_volume_block: Starting block (for pool volumes only)
        -S snap_id: Snapshot ID (for APFS only)
        -v: verbose to stderr
        -V: Print version
        -k password: Decryption password for encrypted volumes
vboxuser@Df1:~/forensics$ icat testimage.dd 12 > recovered_secret.txt
vboxuser@Df1:~/forensics$ fls -r testimage.dd
d/d 11: lost+found
r/r 12: secret.txt
V/V 25601:      $OrphanFiles
+ -/r * 13:     OrphanFile-13
vboxuser@Df1:~/forensics$ icat testimage.dd 12 > recovered_secret.txt
vboxuser@Df1:~/forensics$ cat recovered_secret.txt
Forensic Test File
COnfidential Data
vboxuser@Df1:~/forensics$ sudo mount -o loop testimage.dd /mnt/testimage
vboxuser@Df1:~/forensics$ sudo rm /mnt/testimage/secret.txt
vboxuser@Df1:~/forensics$ sudo umount /mnt/testimage
vboxuser@Df1:~/forensics$ fls -r testimage.dd
d/d 11: lost+found
V/V 25601:      $OrphanFiles
+ -/r * 12:     OrphanFile-12
+ -/r * 13:     OrphanFile-13
vboxuser@Df1:~/forensics$ cat recovered_secret.txt
Forensic Test File
COnfidential Data
vboxuser@Df1:~/forensics$
```

Figure 34 FLS command to show the deleted file and using icat function to retrieve the file

The inode number was then supplied to the icat command, which reconstructed the file successfully from disk image. This explains the forensic principle of file deletion in EXT4 does not instantly destroy data, but marks space as available for reuse.

```
vboxuser@Df1: ~/forensics
        -b dev_sector_size: The size (in bytes) of the device sectors
        -f fstype: File system type (use '-f list' for supported types)
        -o imgoffset: The offset of the file system in the image (in sectors)
        -P pooltype: Pool container type (use '-P list' for supported types)
        -B pool_volume_block: Starting block (for pool volumes only)
        -S snap_id: Snapshot ID (for APFS only)
        -v: verbose to stderr
        -V: Print version
        -k password: Decryption password for encrypted volumes
vboxuser@Df1:~/forensics$ icat testimage.dd 12 > recovered_secret.txt
vboxuser@Df1:~/forensics$ fls -r testimage.dd
d/d 11: lost+found
r/r 12: secret.txt
V/V 25601:      $OrphanFiles
+ -/r * 13:     OrphanFile-13
vboxuser@Df1:~/forensics$ icat testimage.dd 12 > recovered_secret.txt
vboxuser@Df1:~/forensics$ cat recovered_secret.txt
Forensic Test File
COnfidential Data
vboxuser@Df1:~/forensics$ sudo mount -o loop testimage.dd /mnt/testimage
vboxuser@Df1:~/forensics$ sudo rm /mnt/testimage/secret.txt
vboxuser@Df1:~/forensics$ sudo umount /mnt/testimage
vboxuser@Df1:~/forensics$ fls -r testimage.dd
d/d 11: lost+found
V/V 25601:      $OrphanFiles
+ -/r * 12:     OrphanFile-12
+ -/r * 13:     OrphanFile-13
vboxuser@Df1:~/forensics$ cat recovered_secret.txt
Forensic Test File
COnfidential Data
vboxuser@Df1:~/forensics$ fls -r -m / testimage.dd > bodyfile.txt
vboxuser@Df1:~/forensics$
```

Figure 35 generating body file for timeline capability



```
vboxuser@Df1: ~/forensics
vboxuser@Df1:~/forensics$ fls -r testimage.dd
d/d 11:	lost+found
r/r 12:	secret.txt
V/V 25601:	$OrphanFiles
+ -/r * 13:	OrphanFile-13
vboxuser@Df1:~/forensics$ icat testimage.dd 12 > recovered_secret.txt
vboxuser@Df1:~/forensics$ cat recovered_secret.txt
Forensic Test File
CONfidential Data
vboxuser@Df1:~/forensics$ sudo mount -o loop testimage.dd /mnt/testimage
vboxuser@Df1:~/forensics$ sudo rm /mnt/testimage/secret.txt
vboxuser@Df1:~/forensics$ sudo umount /mnt/testimage
vboxuser@Df1:~/forensics$ fls -r testimage.dd
d/d 11:	lost+found
V/V 25601:	$OrphanFiles
+ -/r * 12:	OrphanFile-12
+ -/r * 13:	OrphanFile-13
vboxuser@Df1:~/forensics$ cat recovered_secret.txt
Forensic Test File
CONfidential Data
vboxuser@Df1:~/forensics$ fls -r -m / testimage.dd > bodyfile.txt
vboxuser@Df1:~/forensics$ mactime -b bodyfile.txt > timeline.txt
Old package separator "'" deprecated at /usr/bin/mactime line 154.
Old package separator "'" deprecated at /usr/bin/mactime line 167.
vboxuser@Df1:~/forensics$ cat timeline.txt
Wed Feb 25 2026 20:52:01    16384 macb d/drwx------ 0        0        11       /lost+found
Wed Feb 25 2026 21:14:13        0 ...b -/rrw-r--r-- 0        0        12       /$OrphanFiles/OrphanFile-12 (deleted)
Wed Feb 25 2026 21:14:21        0 .a.. -/rrw-r--r-- 0        0        12       /$OrphanFiles/OrphanFile-12 (deleted)
Wed Feb 25 2026 21:14:25        0 .a.b -/rrw-r--r-- 0        0        13       /$OrphanFiles/OrphanFile-13 (deleted)
Wed Feb 25 2026 21:15:25        0 m.c. -/rrw-r--r-- 0        0        13       /$OrphanFiles/OrphanFile-13 (deleted)
Wed Feb 25 2026 21:25:41        0 m.c. -/rrw-r--r-- 0        0        12       /$OrphanFiles/OrphanFile-12 (deleted)
vboxuser@Df1:~/forensics$
```

Figure 36 Opening timeline and reconstruction chronological file activity

Until overwritten, the data remains recoverable through inode-based extraction. This validates the effectiveness of metadata-level analysis in forensic investigations and highlights importance of acquiring timely evidence.

Limitations

Wireshark cannot decrypt encrypted HTTPS traffic without key material, thereby limiting the visibility into secure communications. On the other hand, Sleuth Kit requires strong command-line knowledge, which may be complex for beginners. Additionally, metadata-based recovery in EXT4 depends on data not being overwritten, meaning the successful recovery is time-sensitive. These limitations highlight the importance of combining multiple forensic tools to ensure comprehensive evidence analysis.

Conclusion

While Wireshark focuses on the network-level evidence through packet inspection and protocol analysis, Sleuth Kit operates at a filesystem level, by enabling an in-depth metadata examination and recovering deleted files. Together, these tools demonstrate the layered forensic methodology by analysing both volatile network traffic and persistent disk artefacts.

Reference

[1] Combs, G. (2023) *Wireshark User's Guide*. Available at: https://www.wireshark.org/docs/wsug_html_chunked/
[2] Carrier, B. (2023) *Autopsy Digital Forensics Platform Documentation*. Available at: https://www.autopsy.com
[3] Carrier, B. (2023) *The Sleuth Kit Documentation*. Available at: https://www.sleuthkit.org/
[4] Martins de Souza, Cristian & Batista, Daniel. (2025). Foremost-NG: An Open-Source Toolkit for Advanced File Carving and Analysis. 10.5753/sbseg_estendido.2025.12307.